Project 4: SQL

Data Set: https://www.kaggle.com/datasets/ahmedmohamed2003/cafe-sales-dirty-data-for-cleaning-training

Purpose of project:

- Loading Data Into SQL server
- Cleaning the data using SQL itself
- Answering Questions

1. **Creating and using Data Base**

```
1      -- Creating a new database
2 •    CREATE DATABASE Project4;
3 •    USE Project4;
```

2. **Creating table to match the data set columns:**

```
5      -- Creating table to match the data
6 •    CREATE TABLE Sales(Transaction_ID VARCHAR(50), Item CHAR(50), Quantity VARCHAR(50),
7      Price_Per_Unit VARCHAR(50),Total_Spent VARCHAR(50),Payment_Method VARCHAR(100),
8      Location VARCHAR(100), Transaction_Date VARCHAR(50));
```

- Using VARCHAR in places where we should put INT as the data isn't clean

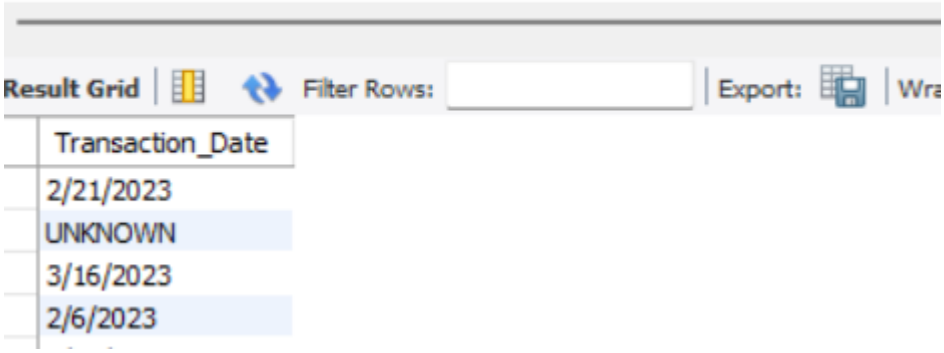
3. **Loading data into server:**

```
10     -- Loading the data set
11 •   LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/dirty_cafe_sales.csv"
12     INTO TABLE Sales
13     FIELDS TERMINATED BY ',' ENCLOSED BY '"'
14     LINES TERMINATED BY '\n'
15     IGNORE 1 ROWS;
```

DATA CLEANING

4. **Identifying the data in column**

```
25      -- Checking Unique Values
26 •    SELECT DISTINCT Transaction_Date FROM Sales;
```



- Not only in this column, but even others had data like:
  - ERROR
  - UNKNOWN
  - BLANK

5. **Adding new column and replacing data from old column and dropping old column**

```
56 •    ALTER TABLE Sales ADD COLUMN T_Date DATE;
57 •    UPDATE Sales SET T_Date= CASE
58      WHEN Transaction_Date NOT LIKE '%ERROR%'
59      AND Transaction_Date NOT LIKE '%UNKNOWN%'
60      AND Transaction_Date !=''
61      THEN STR_TO_DATE(Transaction_Date,'%m/%d/%Y')
62      END;
```

- Using STR_T0_Date for converting the rest are null.

```
64 •    ALTER TABLE Sales DROP COLUMN Transaction_Date;
```

6. **Doing the same for other columns:**

```
78      -- For item column without creating new column
79 •    UPDATE Sales SET Item= REPLACE(Item,"ERROR","");
80 •    UPDATE Sales SET Item= REPLACE(Item,"UNKNOWN","");
```

```
82      -- Updating the Quantity Colum
83 •    UPDATE Sales SET Quantity= REPLACE(Quantity,"ERROR","");
84 •    UPDATE Sales SET Quantity= REPLACE(Quantity,"UNKNOWN","");


88      -- Updating the Price_Per_Unit column
89 •    UPDATE Sales SET Price_Per_Unit=REPLACE(Price_Per_Unit,"ERROR","");
90 •    UPDATE Sales SET Price_Per_Unit=REPLACE(Price_Per_Unit,"UNKNOWN","");


94      -- Updating the Total_Spent column
95 •    UPDATE Sales SET Total_Spent=REPLACE(Total_Spent,"ERROR","");
96 •    UPDATE Sales SET Total_Spent=REPLACE(Total_Spent,"UNKNOWN","");


98      -- Updating the Payment_Method column
99 •    UPDATE Sales SET Payment_Method=REPLACE(Payment_Method,"ERROR","");
100 •   UPDATE Sales SET Payment_Method=REPLACE(Payment_Method,"UNKNOWN","");


102     -- Updating the location column
103 •   UPDATE Sales SET Location=REPLACE(Location,"ERROR","");
104 •   UPDATE Sales SET Location=REPLACE(Location,"UNKNOWN","");
```

7. **Changing data types:**

```
106     -- Changing data types with new columns:
107 •   ALTER TABLE Sales ADD Qty INT;
108 •   UPDATE Sales SET Qty=CASE
109     WHEN Quantity!=''
110     THEN CAST(Quantity AS FLOAT)
111     END;
```

- Changing this to float as INT allows float using CAST

```
ALTER TABLE Sales DROP COLUMN Quantity;
```

8. Doing the same for other columns:

```
115  ALTER TABLE Sales ADD COLUMN Spent_Total INT;
116  UPDATE Sales SET Spent_Total= CASE
117  WHEN Total_Spent!=''
118  THEN CAST(Total_Spent AS FLOAT)
119  END;
120  ALTER TABLE Sales DROP COLUMN Total_Spent;
```

```
122  -- Next:
123  ALTER TABLE Sales ADD COLUMN Unit_Cost INT;
124  UPDATE Sales SET Unit_Cost= CASE
125  WHEN Price_Per_Unit!=''
126  THEN  CAST(Price_Per_Unit AS FLOAT)
127  END;
128  ALTER TABLE Sales DROP COLUMN Price_Per_Unit;
```

```
131  -- Next
132  ALTER TABLE Sales ADD COLUMN Pay_Method CHAR(50);
133  UPDATE Sales SET Pay_Method= CASE
134  WHEN Payment_Method!=''
135  THEN CAST(Payment_Method AS CHAR)
136  END;
137  ALTER TABLE Sales DROP COLUMN Payment_Method;
```

```
139  -- Next
140  ALTER TABLE Sales ADD COLUMN Loc CHAR(100);
141  UPDATE Sales SET Loc=CASE
142  WHEN Location!=''
143  THEN CAST(Location AS CHAR)
144  END;
145  ALTER TABLE Sales DROP COLUMN Location;
```

```
147   -- Next
148 • ALTER TABLE Sales ADD COLUMN Items CHAR(50);
149 • UPDATE Sales SET Items= CASE
150   WHEN Item!=''
151   THEN CAST(Item AS CHAR)
152   END;
153 • ALTER TABLE Sales DROP COLUMN Item;
```

9. Retrieving finalized data

```
156   -- Finalized
157 • SELECT * FROM Sales;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Transaction_ID	T_Date	Qty	Spent_Total	Unit_Cost	Pay_Method	Loc	Items
TXN_1961373	2023-09-08	2	4	2	Credit Card	Takeaway	Coffee
TXN_4977031	2023-05-16	4	12	3	Cash	In-store	Cake
TXN_4271903	2023-07-19	4	NULL	1	Credit Card	In-store	Cookie
TXN_7034554	2023-04-27	2	10	5	NULL	NULL	Salad
TXN_3160411	2023-06-11	2	4	2	Digital Wallet	In-store	Coffee
TXN_2602893	2023-03-31	5	20	4	Credit Card	NULL	Smoothie
TXN_4433211	2023-10-06	3	9	3	NULL	Takeaway	NULL

QUESTIONS

10. Find all transactions that occurred on a specific date. (20th July 2023)

```
162 • SELECT * FROM Sales WHERE T_Date='2023-07-20';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Transaction_ID	T_Date	Qty	Spent_Total	Unit_Cost	Pay_Method	Loc	Items
TXN_8818168	2023-07-20	4	12	NULL	Digital Wallet	NULL	Juice
TXN_9497634	2023-07-20	3	9	3	Cash	NULL	Juice
TXN_6129205	2023-07-20	5	20	4	Cash	In-store	Sandwich
TXN_6802372	2023-07-20	3	9	3	NULL	In-store	Juice
TXN_6009395	2023-07-20	4	16	4	Cash	In-store	Sandwich
TXN_5485774	2023-07-20	2	4	2	Credit Card	NULL	Coffee
TXN_2468359	2023-07-20	2	10	5	NULL	NULL	NULL

11. Count the number of transactions made at each location.

```
165 •  SELECT Loc,COUNT(Loc) AS COUNT FROM Sales GROUP BY Loc
166    HAVING COUNT(Loc);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Loc	COUNT
Takeaway	3022
In-store	3017

12. Calculate the total revenue generated (sum of Total_Spent).

```
169 •  SELECT SUM(Spent_Total) FROM Sales;
```

Result Grid | Filter Rows: | Expor

SUM(Spent_Total)
84872

13. List all unique items sold.

```
172 •  SELECT DISTINCT Items FROM Sales;
```

Result Grid | Filter Rows: | Ex

Items
Coffee
Cake
Cookie
Salad
Smoothie
NULL
Sandwich

14. Find the most frequently purchased item

```
175 •  SELECT Items,COUNT(T_Date) FROM Sales GROUP BY Items
176    HAVING COUNT(T_Date) ORDER BY COUNT(T_DATE) DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cont

Items	COUNT(T_Date)
Juice	1124
Coffee	1123
Salad	1099
Cake	1082
Sandwich	1075
Smoothie	1048

15. Calculate the average Price_per_unit for all items.

```
179 •  SELECT Items, AVG(Unit_Cost) FROM Sales GROUP BY Items
180    HAVING AVG(Unit_Cost);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content

Items	AVG(Unit_Cost)
Coffee	2.0000
Cake	3.0000
Cookie	1.0000
Salad	5.0000
Smoothie	4.0000
NULL	2.9934

16. List all transactions where more than 5 units of an item were sold.

```
183 •  SELECT * FROM Sales WHERE Qty>5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Transaction_ID	T_Date	Qty	Spent_Total	Unit_Cost	Pay_Method	Loc	Items

- **No transactions**

17. Retrieve all records where the Total_Spent is greater than $20

```
186 • SELECT * FROM Sales WHERE Spent_Total>20;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Transaction_ID	T_Date	Qty	Spent_Total	Unit_Cost	Pay_Method	Loc	Items
TXN_2548360	2023-11-07	5	25	5	Cash	Takeaway	Salad
TXN_6342161	2023-01-08	5	25	5	NULL	Takeaway	Salad
TXN_8914892	2023-03-15	5	25	5	Digital Wallet	NULL	NULL
TXN_5220895	2023-06-10	5	25	5	Cash	In-store	Salad
TXN_9517146	2023-10-30	5	25	5	Cash	Takeaway	NULL

18. Find the total quantity of items sold for each Payment_Method

```
189 • SELECT Pay_Method, SUM(Qty) From Sales GROUP BY Pay_Method
190   HAVING SUM(Qty);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Pay_Method	SUM(Qty)
Credit Card	6554
Cash	6533
NULL	9100
Digital Wallet	6647

19. Identify the location with the highest total sales.

```
193 • WITH Tot AS (
194   SELECT Loc, SUM(Spent_Total) AS Total_Spent FROM Sales GROUP BY Loc
195   )
196   SELECT Loc, Total_Spent FROM Tot ORDER BY Total_Spent DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Loc	Total_Spent
NULL	33669
In-store	25939
Takeaway	25264

- **We don't have to use sub query here, but I just wanted to use it.**

20. Find the item with the highest revenue (consider Total_Spent).

```
199 •  SELECT Items, SUM(Spent_Total) FROM Sales GROUP BY Items
200    HAVING SUM(Spent_Total) ORDER BY SUM(Spent_Total) DESC LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fet

Items	SUM(Spent_Total)
Salad	16605

21. List all transactions that occurred in July

```
203 •  SELECT * FROM Sales WHERE T_Date BETWEEN '2023-07-01' AND '2023-07-31';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Transaction_ID	T_Date	Qty	Spent_Total	Unit_Cost	Pay_Method	Loc	Items
TXN_4271903	2023-07-19	4	NULL	1	Credit Card	In-store	Cookie
TXN_4717867	2023-07-28	5	15	3	NULL	Takeaway	NULL
TXN_6855	17	4	12	3	NULL	In-store	NULL
TXN_6420335	2023-07-16	1	2	2	Cash	Takeaway	Coffee
TXN_8779771	2023-07-25	4	8	2	Cash	In-store	Coffee
TXN_2181545	2023-07-14	4	12	3	Credit Card	Takeaway	Juice

TXN_4717867

22. Calculate the total revenue generated per day.

```
206 •  SELECT T_Date, SUM(Spent_Total) FROM Sales GROUP BY T_Date
207    HAVING SUM(Spent_Total) ORDER BY T_Date ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

T_Date	SUM(Spent_Total)
NULL	3967
2023-01-01	168
2023-01-02	163
2023-01-03	140
2023-01-04	265
2023-01-05	265

23. **Rank locations by their total revenue in descending order.**

```
210 • WITH Ran AS(
211   SELECT Loc, SUM(Spent_Total) AS Total FROM Sales GROUP BY Loc
212   )
213   SELECT Loc, Total , RANK()
214   OVER(ORDER BY Total DESC) AS RANKS FROM Ran;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Loc	Total	RANKS
NULL	33669	1
In-store	25939	2
Takeaway	25264	3

JOINS

24. **Breaking the above into two by creating a new table called transactions using two columns of the sales table**

```
218 • CREATE TABLE Transactions AS SELECT Transaction_ID, T_Date FROM Sales;
```

25. **Retrieving all transactions data**

```
224 • SELECT * FROM Transactions;
```

Result Grid | Filter Rows:

Transaction_ID	T_Date
TXN_1961373	2023-09-08
TXN_4977031	2023-05-16
TXN_4271903	2023-07-19
TXN_7034554	2023-04-27
TXN_3160411	2023-06-11
TXN_2602893	2023-03-31

26. **Keeping transaction ID in Sales and dropping T_Date**

```
221 • ALTER TABLE Sales DROP COLUMN T_Date;
```

27. Combine the two tables to retrieve the complete transaction details.

```
228 • SELECT * FROM Sales INNER JOIN Transactions ON
229   Sales.Transaction_ID=Transactions.Transaction_ID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Transaction_ID	Qty	Spent_Total	Unit_Cost	Pay_Method	Loc	Items	Transaction_ID	T_Date
TXN_1961373	2	4	2	Credit Card	Takeaway	Coffee	TXN_1961373	2023-09-08
TXN_4977031	4	12	3	Cash	In-store	Cake	TXN_4977031	2023-05-16
TXN_4271903	4	NULL	1	Credit Card	In-store	Cookie	TXN_4271903	2023-07-19
TXN_7034554	2	10	5	NULL	NULL	Salad	TXN_7034554	2023-04-27
TXN_3160411	2	4	2	Digital Wallet	In-store	Coffee	TXN_3160411	2023-06-11
TXN_2602893	5	20	4	Credit Card	NULL	Smoothie	TXN_2602893	2023-03-31

28. Find transactions where the Payment_Method is missing in one of the tables

```
232 • SELECT * FROM Sales RIGHT JOIN Transactions
233   ON Sales.Transaction_ID=Transactions.Transaction_ID
234   WHERE Sales.Pay_Method IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Transaction_ID	Qty	Spent_Total	Unit_Cost	Pay_Method	Loc	Items	Transaction_ID	T_Date
TXN_7034554	2	10	5	NULL	NULL	Salad	TXN_7034554	2023-04-27
TXN_4433211	3	9	3	NULL	Takeaway	NULL	TXN_4433211	2023-10-06
TXN_4717867	5	15	3	NULL	Takeaway	NULL	TXN_4717867	2023-07-28
TXN_2064365	5	20	4	NULL	In-store	Sandwich	TXN_2064365	2023-12-31
TXN_9437049	5	5	1	NULL	Takeaway	Cookie	TXN_9437049	2023-06-01

29. List items sold at specific locations by combining data from both tables

```
237 • SELECT * FROM Sales INNER JOIN Transactions
238   ON Sales.Transaction_ID=Transactions.Transaction_ID
239   WHERE Sales.Loc='Takeaway';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Transaction_ID	Qty	Spent_Total	Unit_Cost	Pay_Method	Loc	Items	Transaction_ID	T_Date
TXN_1961373	2	4	2	Credit Card	Takeaway	Coffee	TXN_1961373	2023-09-08
TXN_4433211	3	9	3	NULL	Takeaway	NULL	TXN_4433211	2023-10-06
TXN_4717867	5	15	3	NULL	Takeaway	NULL	TXN_4717867	2023-07-28
TXN_2548360	5	25	5	Cash	Takeaway	Salad	TXN_2548360	2023-11-07
TXN_3051279	2	8	4	Credit Card	Takeaway	Sandwich	TXN_3051279	NULL
TXN_9437049	5	5	1	NULL	Takeaway	Cookie	TXN_9437049	2023-06-01

30. Identify items that have no corresponding sales in the second table

```
242 •  SELECT Sales.Items, Transactions.Transaction_ID FROM Sales
243    INNER JOIN Transactions
244    ON Sales.Transaction_ID=Transactions.Transaction_ID
245    WHERE Sales.Spent_Total IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Items	Transaction_ID
Cookie	TXN_4271903
Smoothie	TXN_7958992
NULL	TXN_8927252
Tea	TXN_6650263
Sandwich	TXN_4987129
Juice	TXN_6289610
Juice	TXN_8495063

31. Calculate the total revenue for each location by joining the two tables

```
248 •  SELECT Sales.Loc, SUM(Sales.Spent_Total) FROM Sales
249    INNER JOIN Transactions
250    ON Sales.Transaction_ID=Transactions.Transaction_ID
251    GROUP BY Sales.Loc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Loc	SUM(Sales.Spent_Total)
Takeaway	25264
In-store	25939
NULL	33669